

Design and Comparative Evaluation of RCA, CLA, and CSLA Architectures in a 32-bit RISC-V ALU

Wu Yu-Lun

Department of Electronic Engineering

National Taipei University of Technology

September 2026

Abstract

This project presents the design and comparison of three adder architectures, Ripple Carry Adder (RCA), Carry Lookahead Adder (CLA), and Carry Select Adder (CSLA), in a 32-bit RISC-V ALU. The ALU supports eight operations: ADD, SUB, AND, OR, XOR, SLT, SLL, and SRL. All three versions were implemented in Verilog and verified using Icarus Verilog and GTKWave. Yosys and ABC were used for synthesis and structural logic-depth analysis.

The results show that the RCA has the lowest mapped cell count with 1148 cells, but also the largest logic depth of 74. The CLA has the highest mapped cell count with 1240 cells, but achieves the smallest logic depth of 28. The CSLA provides an intermediate result with 1208 cells and a logic depth of 33. The comparison shows how the adder architecture changes the hardware cost and structural logic depth of the same ALU design.

1. Introduction

The arithmetic logic unit (ALU) is an important part of a processor. It is responsible for arithmetic, logical, comparison, and shift operations. In a 32-bit RISC-V processor, common ALU operations include addition, subtraction, bitwise logic, set-less-than comparison, and shifting. [1]

Among these operations, addition is especially important because the same adder can also be used for subtraction. Therefore, the adder architecture can affect both the hardware cost and the combinational path of the ALU.

The three adder architectures considered here handle carry propagation in different ways. A Ripple Carry Adder (RCA) has a simple structure, but the carry signal must pass through each stage. A Carry Lookahead Adder (CLA) uses generate and propagate signals to calculate carry signals with less carry dependency. A Carry

Select Adder (CSLA) calculates possible results for different carry-in values in advance and selects the correct result when the actual carry becomes available.

In this project, a 32-bit RISC-V ALU supporting ADD, SUB, AND, OR, XOR, SLT, SLL, and SRL is implemented in Verilog. Three versions of the ALU are built using RCA, CLA, and CSLA as the adder architecture. The designs are verified using Icarus Verilog and GTKWave. Yosys and ABC are then used to compare the mapped cell count and structural logic depth of the three designs.

This work compares the hardware cost and logic depth of RCA, CLA, and CSLA when they are used in the same 32-bit ALU. The comparison focuses on the following question:

How do RCA, CLA, and CSLA architectures affect the mapped hardware cost and structural logic depth of a 32-bit RISC-V ALU?

2. Background and ALU Architecture

The ALU implemented in this project is a 32-bit combinational circuit. It has two 32-bit inputs, A and B, and produces a 32-bit result. A 3-bit control signal, ALU_Control, is used to select one of eight supported operations.

The supported operations are ADD, SUB, AND, OR, XOR, SLT, SLL, and SRL. Their control codes are listed in Table 1.

Addition and subtraction use the same arithmetic unit. For subtraction, the B input is inverted and the carry-in of the adder is set to 1. This allows subtraction to be performed using two's complement arithmetic:

$$A - B = A + (\sim B) + 1$$

AND, OR, and XOR are implemented as bitwise logic operations. SLT performs a signed comparison between A and B. For the shift operations, the lower five bits of B are used as the shift amount, since a 32-bit value only requires shift amounts from 0 to 31.

The outputs from the arithmetic, logic, comparison, and shift units are sent to the result multiplexer. ALU_Control determines which value is selected as the final result. Figure 1 shows the overall architecture of the 32-bit ALU.

Table 1. ALU control encoding and supported operations.

ALU_Control	Operation	Description
000	ADD	$A + B$
001	SUB	$A - B$
010	AND	Bitwise AND
011	OR	Bitwise OR
100	XOR	Bitwise XOR
101	SLT	Signed set-less-than
110	SLL	Logical left shift
111	SRL	Logical right shift

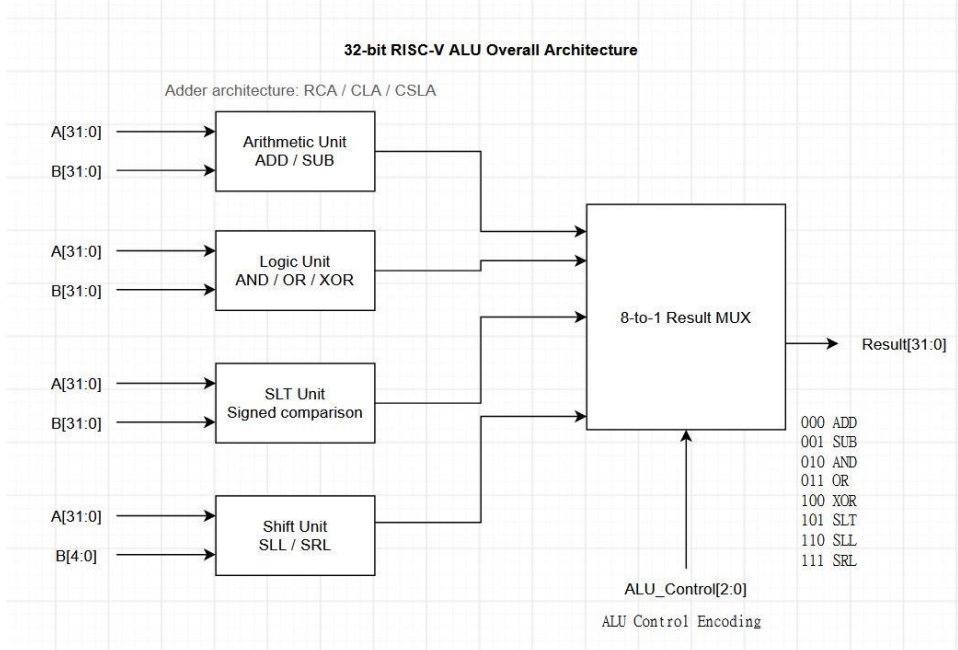


Figure 1. 32-bit RISC-V ALU overall architecture.

3. Adder Architectures

This section introduces the three adder architectures used in the ALU: Ripple Carry Adder (RCA), Carry Lookahead Adder (CLA), and Carry Select Adder (CSLA). The main difference between these architectures is how the carry signal is generated and propagated. Since the adder is shared by ADD and SUB operations, its structure can affect the hardware cost and logic depth of the ALU. [2]

3.1 Ripple Carry Adder

A Ripple Carry Adder is one of the simplest adder architectures. It is built by connecting multiple full adders in series. The carry-out from each stage is used as the carry-in of the next stage.

In a 32-bit RCA, the carry may need to propagate through all 32 full-adder stages before the final result becomes available. This structure is simple and requires relatively little additional control logic, but the long carry path can increase the logic depth.

The 32-bit RCA used here consists of 32 full adders and serves as the baseline for comparison with the CLA and CSLA. Figure 2 shows the basic structure of a 4-bit RCA.

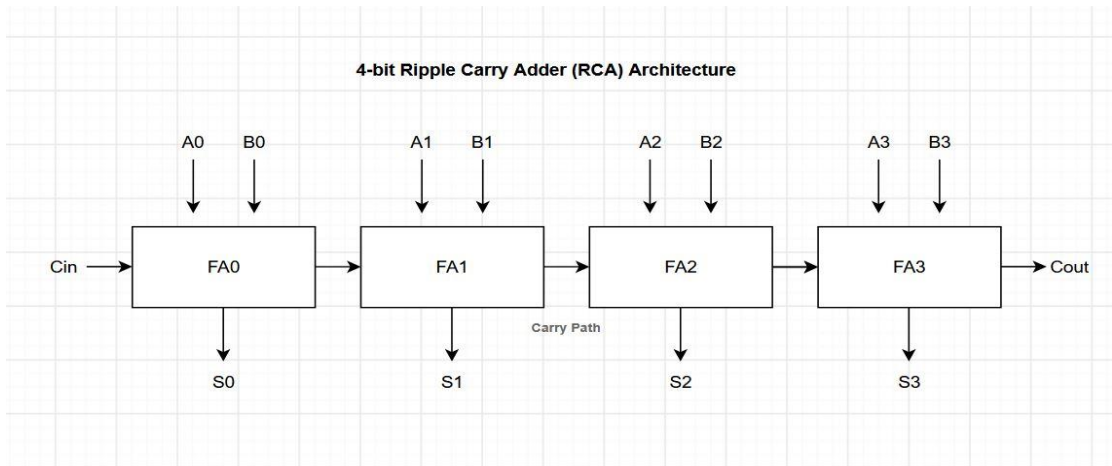


Figure 2. 4-bit Ripple Carry Adder (RCA) architecture.

3.2 Carry Lookahead Adder

A Carry Lookahead Adder reduces the long carry propagation found in an RCA by calculating carry signals from generate and propagate terms. For each bit, the propagate and generate signals are defined as:

$$p_i = A_i \oplus B_i$$

$$g_i = A_i \wedge B_i$$

Using these signals, the carry values can be calculated from the input signals instead of waiting for the carry to pass through every full-adder stage.

The 32-bit CLA is built from eight 4-bit CLA blocks. Each 4-bit block generates group propagate and group generate signals, which are used to determine the carry input of the following blocks. This structure reduces the carry dependency compared with the RCA, but it requires additional logic for carry generation.

Figure 3 shows the structure of the 4-bit CLA used in this project.

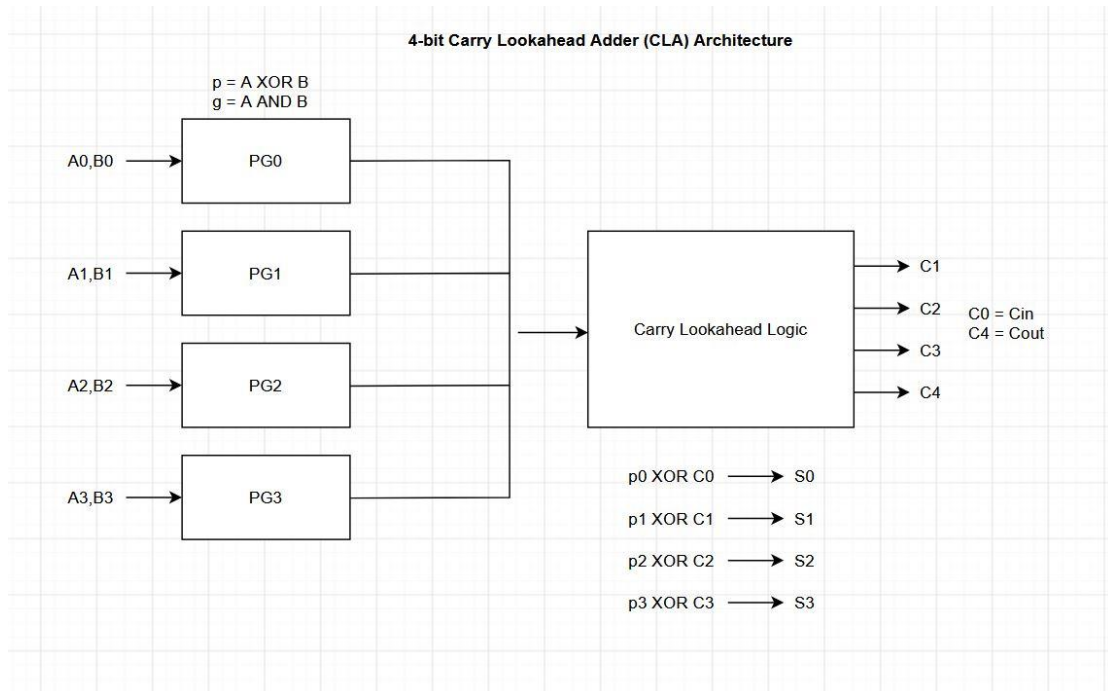


Figure 3. 4-bit Carry Lookahead Adder (CLA) architecture.

3.3 Carry Select Adder

A Carry Select Adder reduces the dependency on carry propagation by computing two possible results in parallel. One path assumes that the carry-in is 0, while the other path assumes that the carry-in is 1. When the actual carry-in becomes available, a multiplexer selects the correct result.

Each 4-bit CSLA block contains two 4-bit Ripple Carry Adders. The two adders calculate the sum and carry-out for $C_{in} = 0$ and $C_{in} = 1$ at the same time. [3] The actual carry-in is then used as the select signal for the output multiplexers. The 32-bit CSLA uses eight 4-bit CSLA blocks. Compared with the RCA, this structure can reduce carry propagation dependency, but it requires additional hardware because both carry-in cases are calculated in parallel.

Figure 4 shows the structure of the 4-bit CSLA used in this project.

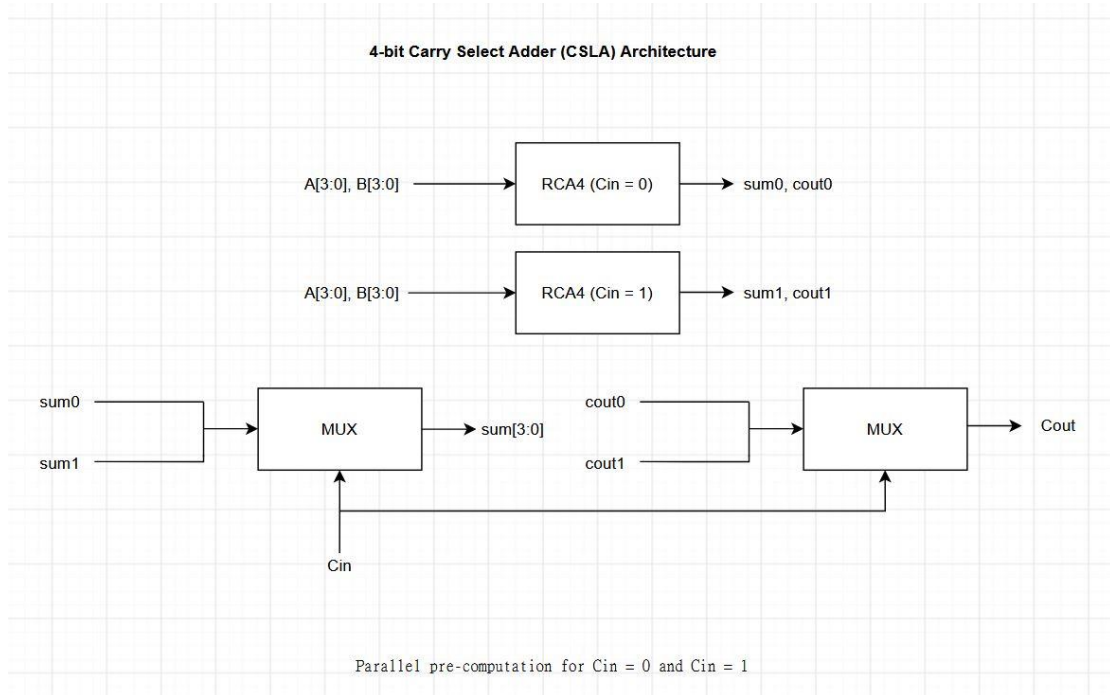


Figure 4. 4-bit Carry Select Adder (CSLA) architecture.

4. Implementation and Verification Methodology

The three ALU designs were implemented in Verilog and tested using the same functional test cases. The main difference between the three versions is the adder architecture used in the arithmetic unit. The remaining ALU functions, including AND, OR, XOR, SLT, SLL, and SRL, were kept the same in order to make the comparison fair.

The RCA implementation uses a 32-stage full-adder carry chain. The CLA implementation is organized as eight 4-bit CLA blocks, while the CSLA implementation uses eight 4-bit blocks that pre-compute results for $Cin = 0$ and $Cin = 1$. The same top-level ALU logic was kept unchanged so that only the adder architecture varied between the three designs.

Functional simulation was performed using Icarus Verilog. Testbenches were created to verify both the individual adder modules and the complete 32-bit ALU. The complete ALU testbench exercised all eight ALU control codes, including arithmetic, logic, signed SLT, and shift operations. GTKWave was used to inspect the simulation waveforms and confirm that the ALU outputs matched the expected results for different control signals.

After functional verification, the designs were synthesized using Yosys [4]. ABC [5] was used for logic mapping and logic-depth analysis. The same synthesis flow and gate set were applied to all three ALU versions.

Two metrics were used for comparison. The mapped cell count was used as a hardware-cost proxy, while the ABC logic level was used as a structural logic-depth proxy. The

reported logic depth should not be interpreted as an actual delay in nanoseconds because no standard-cell timing library or physical implementation was used.

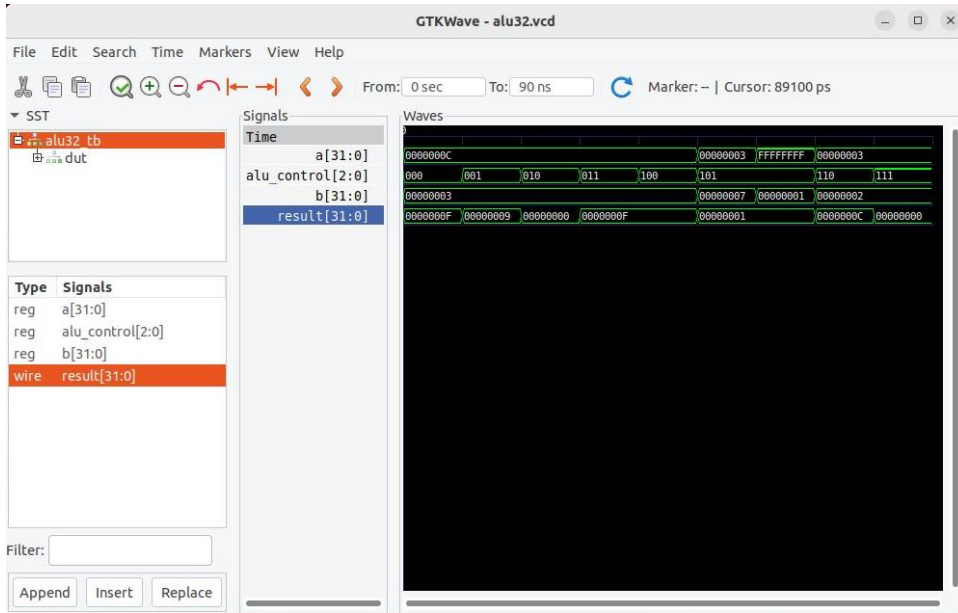


Figure 5. Functional simulation waveform of the 32-bit ALU.

5. Results and Discussion

The synthesis results of the three ALU implementations are summarized in Table 2. Since the same ALU functions and synthesis flow were used for all three designs, the comparison mainly reflects the effect of changing the adder architecture.

Table 2. Synthesis comparison of the three ALU implementations.

Architecture	Mapped Cells	Logic Depth
RCA	1148	74
CSLA	1208	33
CLA	1240	28

RCA gives the lowest mapped cell count at 1148 cells, but its logic depth reaches 74. Replacing it with CSLA adds only 60 cells, increasing the mapped cell count by about 5.2%, while reducing the logic depth to 33. CLA reduces the logic depth further to 28, although its mapped cell count increases to 1240. Compared with RCA, this corresponds to about 8.0% more mapped cells and a 62.2% reduction in structural logic depth. In this comparison, RCA provides the lowest hardware cost, CLA provides the shortest structural path, and CSLA falls between the two.

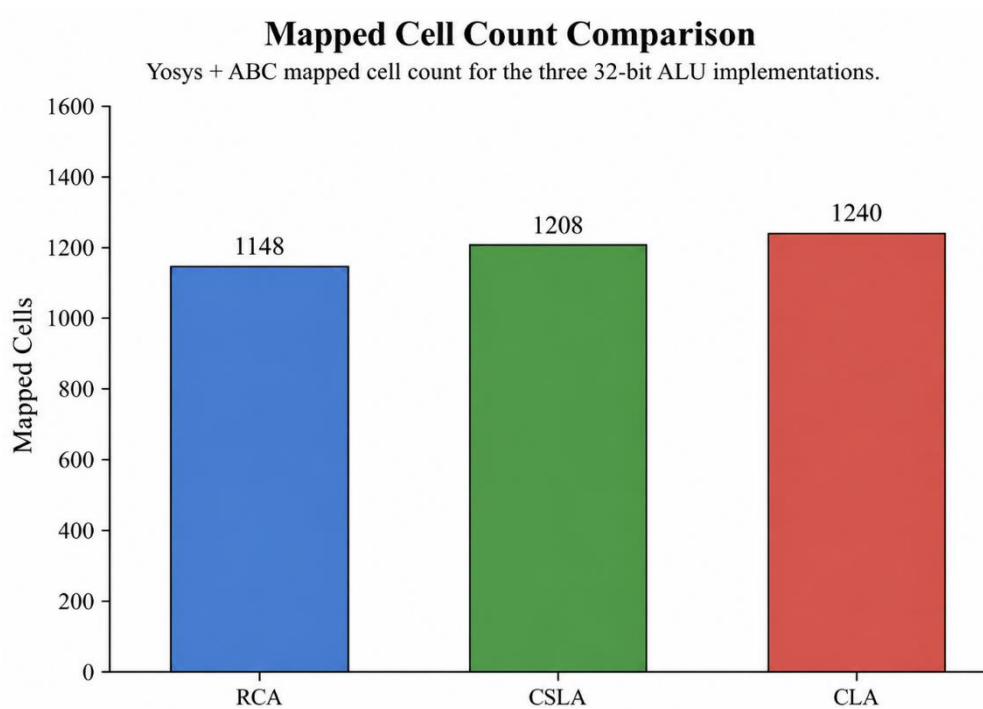


Figure 6. Mapped cell count comparison of the three ALU implementations.

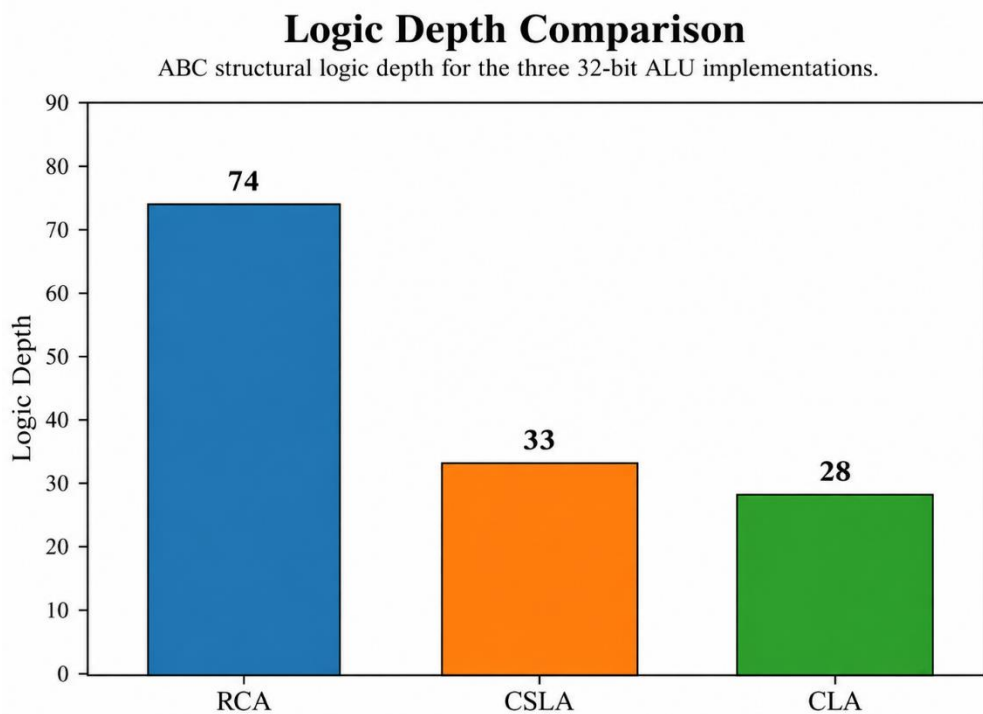


Figure 7. Structural logic depth comparison of the three ALU implementations.

6. Conclusion

This project implemented and compared three adder architectures, RCA, CLA, and CSLA, in the same 32-bit RISC-V ALU. All three designs support the same eight ALU operations

and were verified using the same simulation and synthesis flow.

The synthesis results show that the RCA has the lowest mapped cell count, but also the largest logic depth. The CLA achieves the smallest logic depth, while requiring the largest number of mapped cells. The CSLA provides a middle point between the RCA and CLA in both hardware cost and logic depth.

The logic-depth results in this project are based on ABC structural logic levels rather than actual timing delay in nanoseconds. A useful next step would be to repeat the synthesis with a standard-cell Liberty library. This would allow the three ALU implementations to be compared using actual cell area and timing instead of mapped-cell count and structural logic depth.

References

- [1] RISC-V International, The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, Version 20260120, 2026.
- [2] P. Balasubramanian and N. Mastorakis, “Performance Comparison of Carry-Lookahead and Carry-Select Adders Based on Accurate and Approximate Additions,” *Electronics*, vol. 7, no. 12, p. 369, Dec. 2018, doi: 10.3390/electronics7120369.
- [3] O. J. Bedrij, “Carry-select adder,” *IRE Transactions on Electronic Computers*, vol. EC-11, no. 3, pp. 340–346, Jun. 1962, doi: 10.1109/IRETELC.1962.5407919.
- [4] C. Wolf and J. Glaser, “Yosys—A Free Verilog Synthesis Suite,” in *Proc. Austrochip Workshop on Microelectronics*, 2013, pp. 47–52.
- [5] R. K. Brayton and A. Mishchenko, “ABC: An Academic Industrial-Strength Verification Tool,” in *Computer Aided Verification (CAV 2010)*, Lecture Notes in Computer Science, vol. 6174, 2010, pp. 24–40, doi: 10.1007/978-3-642-14295-6_5.